

DOKUMENTASI SETUP DAN RUN DEMO

Technical Test - Solar Telemetry
Frontend · Backend · Telemetry Simulator · Automation · SQL

Glen Davis Kusuma | Fullstack Developer

Daftar Isi

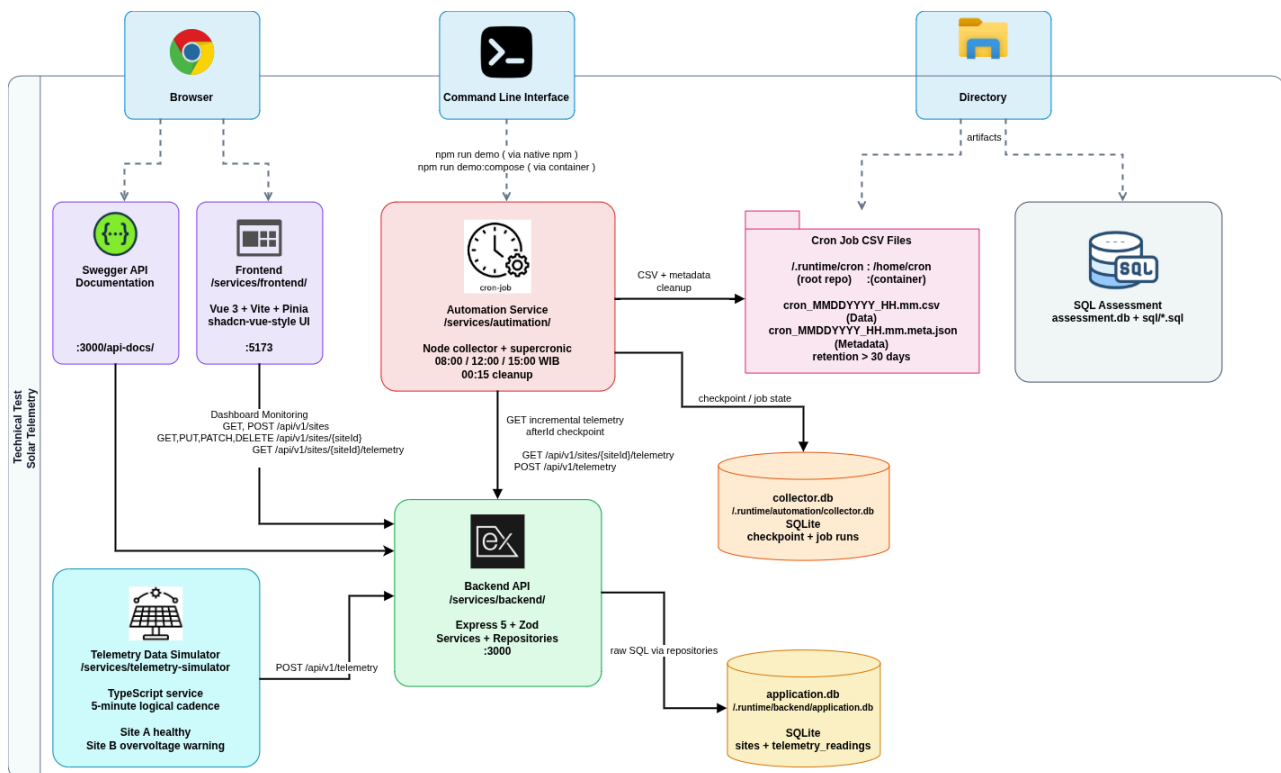
Daftar Isi.....	2
1. Tentang.....	3
2. Arsitektur dan Ringkasan Sistem.....	3
2.1 Frontend.....	4
2.2 Backend API.....	5
2.3 Telemetry Simulator.....	5
2.4 Automation Service.....	6
2.5 Data Processing / SQL Assessment.....	6
3. Demo.....	7
3.1 Demo Aplikasi - npm run demo:compose.....	7
3.2 Demo Automation Cleanup - npm run test:cleanup:compose.....	8
4. Requirement & Quick Setup.....	9
4.1 Minimum software requierments.....	9
4.2 Quick setup.....	9
5. Menjalankan Demo Menggunakan Compose.....	9
5.1 Demo aplikasi keseluruhan.....	9
5.2 Demo cleanup automation.....	10
6. Menjalankan Demo Menggunakan npm.....	10
7. SQL Assessment.....	10

1. Tentang

Technical test meminta backend berbasis Node.js/Express yang mampu menerima data formulir dari frontend, menyimpan data, dan mengembalikan data yang telah tersimpan melalui endpoint yang dirancang dengan baik. Technical test juga meminta proses automation untuk melakukan data collection pada jadwal 08.00, 12.00, dan 15.00 WIB, menyimpan hasil collection dalam bentuk file CSV, serta melakukan data cleansing terhadap file yang berumur lebih dari satu bulan.

Untuk menghubungkan kebutuhan backend dan automation dalam satu alur yang dapat didemonstrasikan, implementasi menggunakan tema Solar Telemetry, yaitu *proof of concept* sistem terpadu untuk memantau, mengumpulkan, dan menampilkan data kinerja Pembangkit Listrik Tenaga Surya (PLTS) dengan data simulasi. Backend berperan sebagai service penerima dan penyimpan data site serta telemetry pembangkit tenaga surya. Frontend menyediakan form pengelolaan situs (site) sekaligus dashboard untuk membaca kembali data yang disimpan backend. Telemetry Simulator menghasilkan data pembacaan dua site secara terprogram yang mensimulasikan produksi PLTS sepanjang 1 hari penuh, sedangkan Automation Service mengambil telemetry baru sesuai jadwal cronjob dari backend dan menghasilkan artefak CSV beserta metadata. Tugas SQL employee tetap dipisahkan dari sistem Solar Telemetry dan disediakan sebagai kumpulan file .sql.

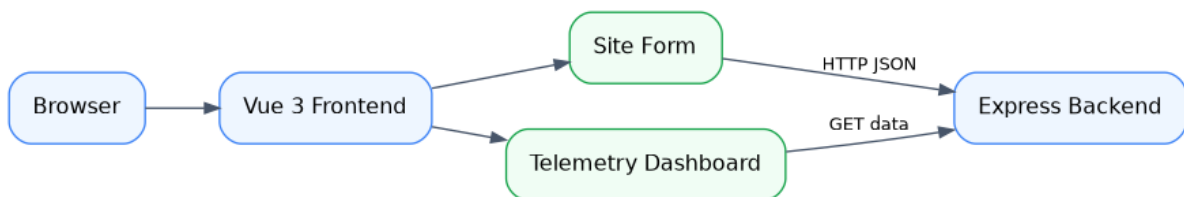
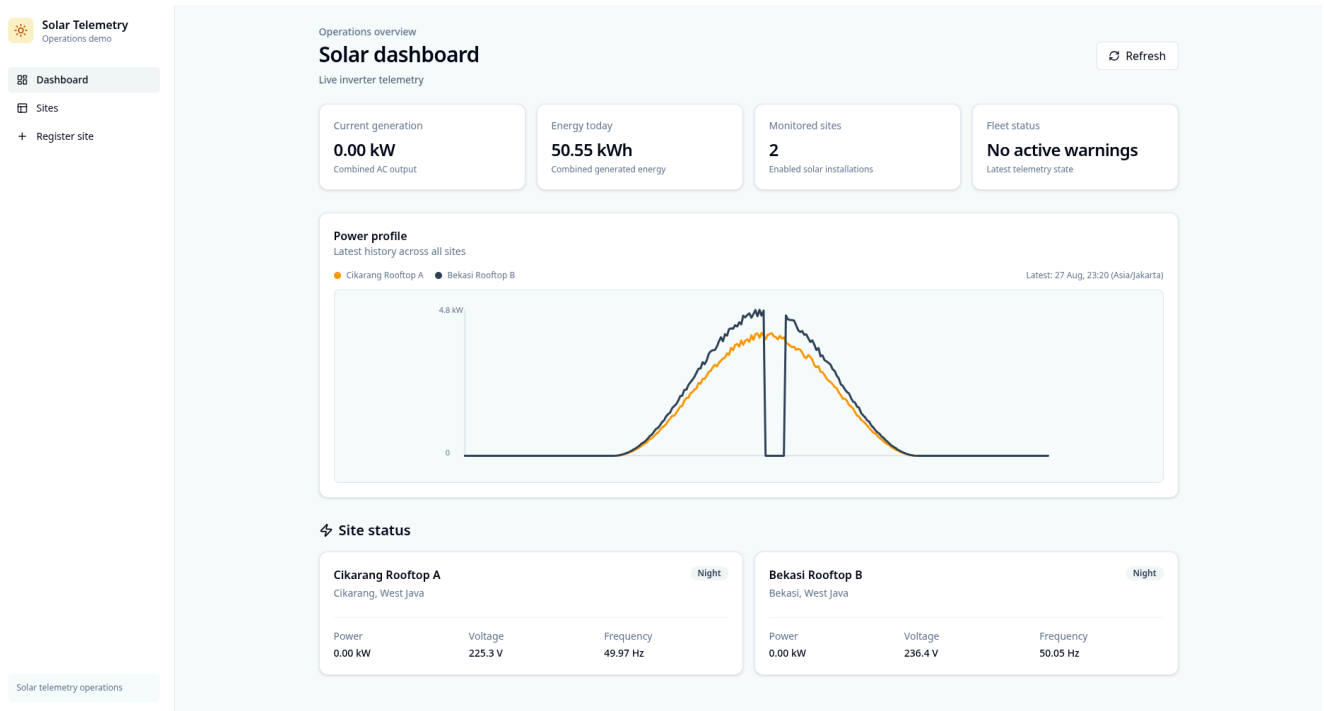
2. Arsitektur dan Ringkasan Sistem



Arsitektur terdiri dari empat service utama: Frontend, Backend API, Telemetry Simulator, dan Automation Service. Browser mengakses Frontend pada port 5173 dan dokumentasi API pada Backend port 3000. Telemetry Simulator dan Automation Service berkomunikasi dengan Backend menggunakan HTTP REST API JSON. Backend menyimpan data site dan telemetry ke SQLite *application.db*, sedangkan Automation Service menyimpan checkpoint dan job state ke *collector.db* serta menulis hasil collection ke *.runtime/cron* pada host atau */home/cron* di container.

Selain akses melalui browser, anda dapat menjalankan demo dari command line, memeriksa file CSV dan metadata melalui file explorer/IDE, serta menjalankan SQL assessment secara terpisah.

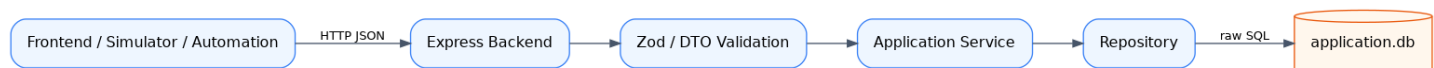
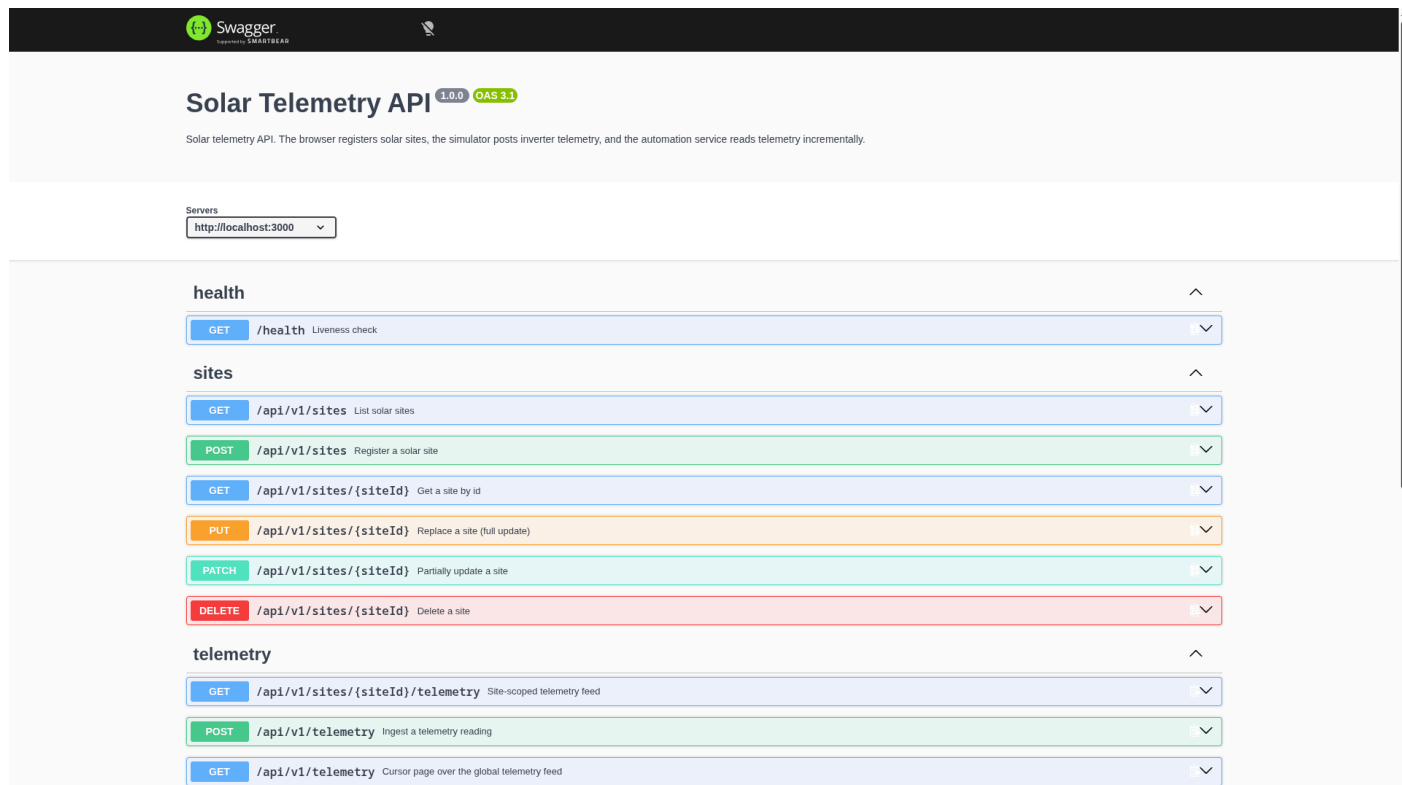
2.1 Frontend



Frontend dibangun untuk memenuhi bagian requirement bahwa backend menerima data formulir dari frontend dan mampu mengembalikan data yang telah disimpan ketika diminta kembali oleh frontend. Form site mengirim data konfigurasi site ke Backend API, sedangkan halaman dashboard dan detail site membaca data site serta telemetry melalui endpoint yang sama.

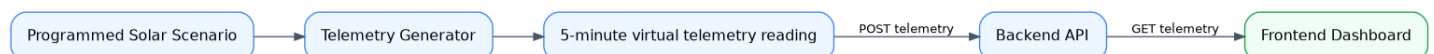
Pada demo, frontend juga digunakan sebagai tampilan observasi. Site A (Cikarang) menampilkan kondisi normal, sedangkan Site B (Bekasi) menampilkan contoh gangguan *GRID_OVERVOLTAGE*. Gangguan ini sengaja disimulasikan agar anda dapat melihat perbedaan status normal dan warning langsung pada dashboard, bukan hanya dari log command line.

2.2 Backend API



Backend merupakan pusat integrasi untuk tiga komponen lain. Frontend menggunakan backend untuk membuat dan membaca site, Telemetry Simulator mengirim pembacaan telemetry ke backend, dan Automation Service meminta telemetry secara incremental berdasarkan checkpoint collection terakhir. Semua request melewati validasi Zod dan application service sebelum data diakses melalui repository. SQLite *application.db* menyimpan *sites* dan *telemetry_readings*. Pemisahan controller, validation/DTO, service, dan repository membuat akses HTTP tidak berhubungan langsung dengan query database dan memudahkan pengujian setiap boundary secara terpisah.

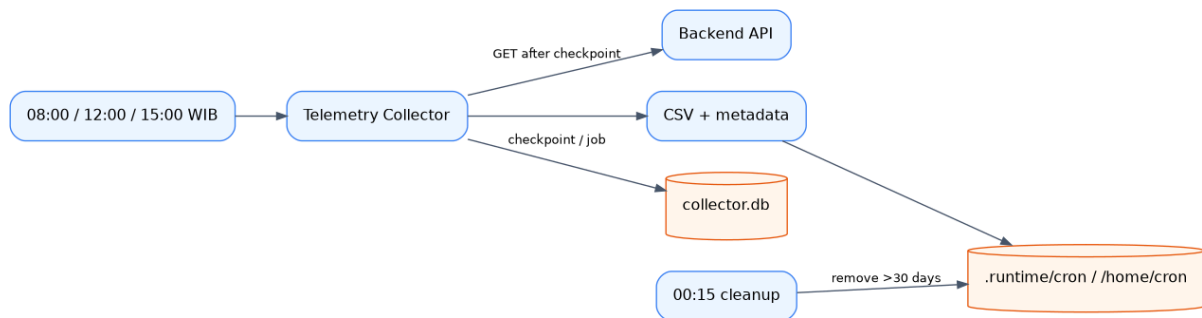
2.3 Telemetry Simulator



Telemetry Simulator menyediakan sumber data yang dibutuhkan untuk menguji penerimaan data backend dan proses automatic data collection. Setiap record simulasi merepresentasikan pembacaan kondisi site pada interval lima menit waktu virtual. Pada demo, dua hari telemetry dipercepat menjadi sekitar dua menit waktu nyata sehingga perubahan produksi, status, dan proses archive dapat diamati tanpa menunggu satu hari penuh.

Dua site memiliki skenario yang berbeda. Site A (cikarang) digunakan sebagai baseline kondisi sehat. Site B (Bekasi) menerima contoh gangguan tegangan lebih pada periode tertentu sehingga backend menyimpan status warning dan frontend dapat menampilkan site yang memerlukan perhatian.

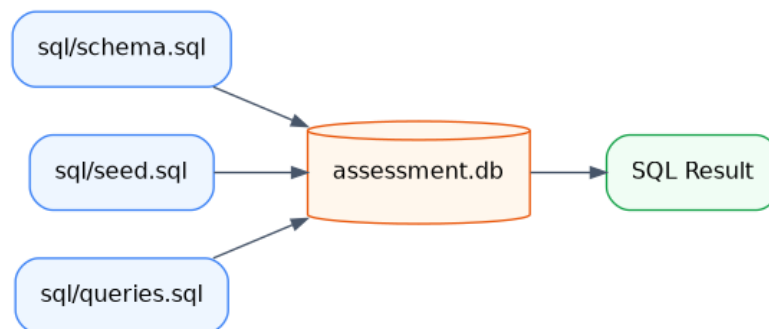
2.4 Automation Service



Automation Service memenuhi requirement collection data terjadwal dan data cleansing. Collector membaca checkpoint terakhir dari collector.db, meminta telemetry yang belum pernah berhasil diarsipkan, kemudian menulis pasangan file `cron_MMDDYYYY_HH.mm.csv` dan `cron_MMDDYYYY_HH.mm.meta.json`. Checkpoint hanya diperbarui setelah artefak berhasil difinalisasi, sehingga kegagalan collection tidak menyebabkan data terlewat.

Jadwal production dijalankan oleh supercronic di container pada 08.00, 12.00, dan 15.00 WIB. Cleanup dijalankan terpisah pada 00.15 dan menghapus artefak generated yang lebih tua dari 30 hari. Pada demo, jadwal collection dipicu secara terprogram pada waktu virtual agar skenario archive dan recovery dapat selesai dalam beberapa menit.

2.5 Data Processing / SQL Assessment

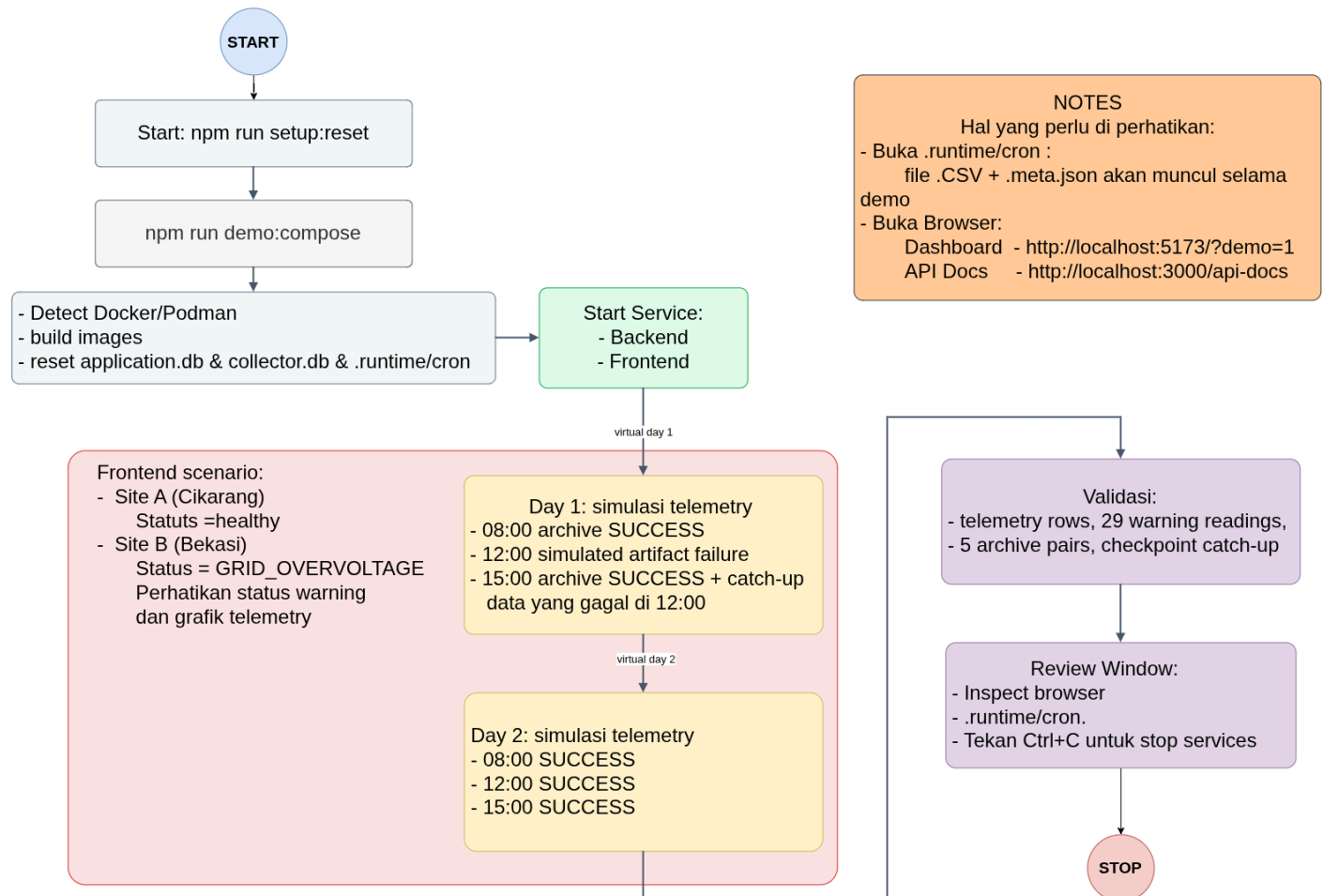


Data processing employee tidak menjadi bagian dari runtime Solar Telemetry. Implementasinya berupa `schema.sql`, `seed.sql`, dan `queries.sql` yang dapat dijalankan pada database assessment terpisah. Dengan pemisahan ini, requirement SQL dapat dinilai tanpa mencampurkan tabel employee dengan database telemetry.

3. Demo

Terdapat dua demo yang dijalankan secara terpisah. Demo aplikasi menunjukkan integrasi Frontend, Backend, Telemetry Simulator, dan Automation Collector. Demo cleanup memfokuskan verifikasi cron configuration dan penghapusan file berumur lebih dari 30 hari. Sebelum menjalankan masing-masing demo, gunakan ***npm run setup:reset*** agar database dan file demo kembali ke baseline yang diketahui.

3.1 Demo Aplikasi - npm run demo:compose



Hal yang perlu diperhatikan selama demo:

- buka dashboard <http://localhost:5173/?demo=1> ;
- buka API documentation <http://localhost:3000/api-docs> bila ingin melihat kontrak endpoint;
- buka folder **.runtime/cron** melalui file explorer atau IDE untuk melihat file CSV dan .meta.json yang terbentuk;
- perhatikan Site B (Bekasi) ketika status berubah menjadi warning **GRID_OVERVOLTAGE**;
- pada Day 1 pukul virtual 12.00, archive sengaja dibuat gagal sebelum finalisasi artefak;
- pada collection Day 1 pukul 15.00, data yang belum berhasil diarsipkan pada pukul 12.00 ikut terambil kembali karena checkpoint tidak berubah;
- Day 2 menjalankan collection 08.00, 12.00, dan 15.00 tanpa simulated archive failure.

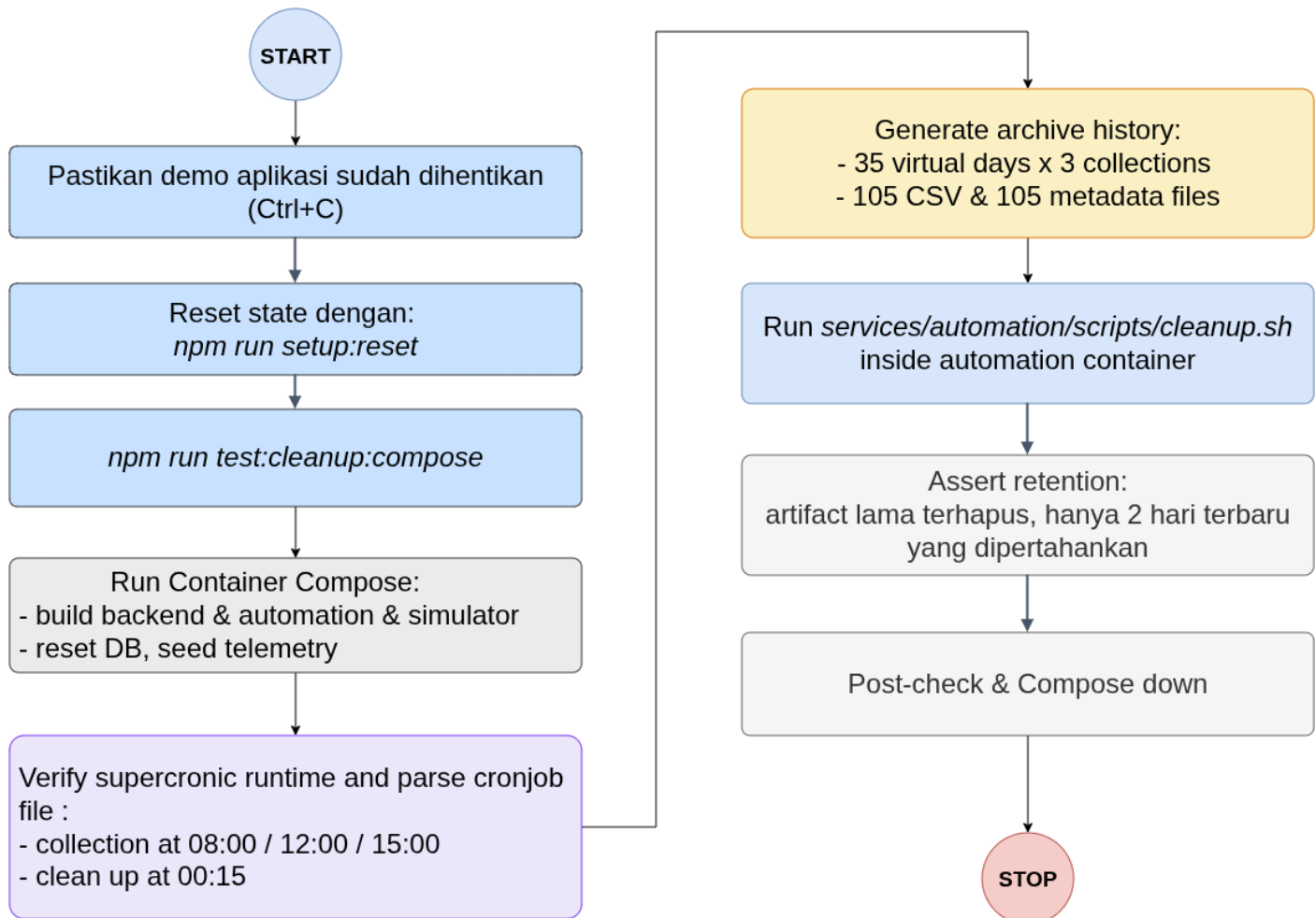
Setelah validasi selesai dan pesan Compose two-day demo Selesai, backend dan frontend tetap berjalan agar anda dapat memeriksa dashboard, API documentation, dan artefak cron. **Tekan Ctrl+C** setelah proses demo selesai, script kemudian akan menjalankan Compose down.

3.2 Demo Automation Cleanup - `npm run test:cleanup:compose`

NOTES

Perhatikan `.runtime/cron/`

Lihat jumlah file yang di generate sampai 35 pasang kemudian file lama agar >30 hari akan di hapus menggunakan script cleanup.



Demo cleanup tidak dijalankan bersamaan dengan demo aplikasi. Hentikan **`npm run demo:compose`** menggunakan **Ctrl+C**, reset state, kemudian jalankan cleanup demo dengan **`npm run setup:reset`**. Script menggunakan Compose project khusus, membuat history artefak untuk 35 hari virtual, mengubah filesystem modification time pada file lama untuk mewakili umur lebih dari 30 hari, menjalankan `cleanup.sh` yang sama dengan automation service, dan memastikan hanya artefak yang masih berada dalam retention window yang dipertahankan.

4. Requirement & Quick Setup

4.1 Minimum software requirements

- [Git](#)
- Node.js 22 dan npm 10+ (<https://git-scm.com/install/>)
- nvm - Node Version Manager (disarankan install <https://github.com/nvm-sh/nvm> untuk memilih versi nodejs 22 secara spesifik)
- browser modern
- [Docker Engine](#) dan [Docker Compose](#) atau [Podman](#) dan [Podman Compose](#) untuk demo yang direkomendasikan
- SQLite CLI (sqlite3) untuk cara termudah menjalankan SQL assessment (<https://sqlite.org/cli.html>)

4.2 Quick setup

```
git clone https://github.com/glenkusuma/20260824-techtest.git
# Atau
unzip 20260824-techtest.zip

cd 20260824-techtest

nvm install 22
nvm use 22
npm ci
npm run setup
```

npm run setup membuat *.env* dari *.env.example* bila belum ada, menyiapkan *.runtime/backend*, *.runtime/automation*, dan *.runtime/cron*, kemudian membuat baseline database demo ketika file database belum tersedia.

Untuk mengembalikan baseline secara sengaja:

```
npm run setup:reset
```

Rekomendasi menjalankan Demo dengan Docker/Podman Compose karena service automation berjalan pada environment yang paling dekat dengan konfigurasi cron sebenarnya.

5. Menjalankan Demo Menggunakan Compose

5.1 Demo aplikasi keseluruhan

Pastikan port 3000 dan 5173 tidak sedang digunakan oleh instance lain.

```
npm run setup:reset
npm run demo:compose
```

Saat demo mulai, buka dua area berikut:

1. Browser: <http://localhost:5173/?demo=1>
 2. File explorer / IDE: *20260824-techtest/.runtime/cron*
- Opsional, buka API documentation di <http://localhost:3000/api-docs>.

Demo menjalankan simulasi dua hari secara virtual. Setelah demo selesai, service tidak langsung berhenti. Anda bisa menggunakan waktu tersebut untuk melihat status site, grafik telemetry, isi CSV, dan metadata archive. **Tekan Ctrl+C** setelah selesai, script kemudian akan menjalankan compose down.

5.2 Demo cleanup automation

Demo ini harus dijalankan setelah demo aplikasi dihentikan.

```
npm run setup:reset  
npm run test:cleanup:compose
```

Sebelum cleanup dijalankan, perhatikan `.runtime/cron` ketika script menghasilkan archive history. Sesudah `cleanup.sh`, script memverifikasi bahwa file lama telah dihapus dan file terbaru tetap ada. Demo cleanup memiliki lifecycle sendiri dan melakukan compose down setelah assertion selesai.

6. Menjalankan Demo Menggunakan npm

Jika Docker/Podman tidak tersedia, demo aplikasi dapat dijalankan langsung dengan Node.js:

```
npm run setup:reset  
npm run demo
```

Backend dan frontend tetap dapat didemonstrasikan, dan collector dipanggil pada waktu virtual oleh demo orchestrator. Namun mode npm-only tidak menjalankan service automation pada daemon cron container, sehingga verifikasi jadwal cronjob (supercronic) dan lifecycle container tidak sekuat mode Compose. Untuk penilaian automation, **`npm run demo:compose`** dan **`npm run test:cleanup:compose`** adalah jalur yang direkomendasikan.

Untuk development interaktif tanpa skenario dua hari:

```
npm run dev
```

Open:

- Frontend: <http://localhost:5173>
- API docs: <http://localhost:3000/api-docs>

7. SQL Assessment

Cara paling sederhana adalah menggunakan SQLite CLI:

```
sqlite3 assessment.db < sql/schema.sql  
sqlite3 assessment.db < sql/seed.sql  
sqlite3 assessment.db < sql/queries.sql
```

Bila sqlite3 CLI tidak tersedia, script dapat dijalankan pada tool/database SQL lain yang kompatibel setelah menyesuaikan syntax DDL atau tipe data bila engine tersebut memerlukannya.